Cryptography Unit III: Block Ciphers (Detailed Notes)

Syllabus: Classical encryption techniques (substitution and transposition ciphers), cryptanalysis, steganography, stream and block ciphers, modern block ciphers (principles, Shannon's confusion and diffusion), Data Encryption Standard (DES), strength of DES, Triple DES, AES. Primary text: Atul Kahate, Cryptography and Network Security.

Part A: Classical Encryption Techniques

Classical ciphers are pen-and-paper schemes. They are not secure today, but they introduce the two building blocks used by every modern cipher: **substitution** (replacing units of plaintext with other units) and **transposition** (rearranging the positions of units). Letters are numbered A = 0, B = 1, ..., Z = 25 in the calculations below.

1. Substitution Ciphers

1.1 Caesar cipher

Each letter is replaced by the letter k places further along the alphabet. Encryption: $C = (P + k) \bmod 26$. Decryption: $P = (C - k) \bmod 26$. Julius Caesar used k = 3.
Example, k = 3: HELLO becomes KHOOR (H→K, E→H, L→O, L→O, O→R).
Weakness: only 25 non-trivial keys, so an attacker can try all of them (brute force) and read the one that gives sensible text. The algorithm is also known, so the key is the only secret.

1.2 Monoalphabetic substitution

Any permutation of the 26 letters is used as the substitution alphabet, giving 26! (about 4×10^{26}) keys, far too many to brute force. Each plaintext letter always maps to the same ciphertext letter, however, so the statistics of the language show through.

- In English the most frequent letters are E (about 12.7 percent), T, A, O, I, N, S, H, R. The least frequent are Z, Q, X, J.
- Common digrams: TH, HE, IN, ER, AN. Common trigrams: THE, AND, ING.
- **Frequency analysis**: count letter frequencies in the ciphertext and match the most frequent ciphertext letter to E, the next to T, and so on, then refine using digrams and word patterns. This is why one-to-one substitution is easy to break for long messages.

1.3 Playfair cipher

The Playfair cipher (Charles Wheatstone, popularized by Lord Playfair) encrypts **pairs of letters (digrams)**, which flattens the frequency pattern compared to single-letter substitution. There are 25 × 25 = 625 digrams, so frequency analysis is much harder.
Key matrix: write a keyword in a 5 × 5 grid, omit repeated letters, then fill the remaining letters of the alphabet in order, with I and J sharing one cell.

Rules

1. Split the plaintext into digrams. If both letters of a pair are the same, insert a filler (usually X) between them. If a single letter remains at the end, add a filler.
2. If both letters fall in the same row, replace each with the letter to its right (the last wraps to the first).
3. If both letters fall in the same column, replace each with the letter below it (the bottom wraps to the top).
4. Otherwise, form a rectangle with the two letters as opposite corners. Replace each letter with the letter in its own row at the column of the other letter.

Decryption reverses the rules: left in the same row, up in the same column, rectangle the same way.

Worked example. Keyword MONARCHY. Matrix:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Plaintext HELLO becomes digrams HE, LX, LO. HE is a rectangle: H→C, E→F. LX is a rectangle: L→S, X→U. LO is a rectangle: L→P, O→M. Ciphertext: CFSUPM.

Two more cases. Same row: BD, where row 2 reads C H Y B D, so B→D and D wraps around to C, giving DC. Same column: OF, where O is in row 1 and F is in row 3 of column 2, so O→H and F→P (each replaced by the letter below it), giving HP.

Weakness: digram frequencies still show through (about 600 digrams, but they are not uniform), and a few hundred letters of ciphertext are usually enough to break it.

1.4 Hill cipher

Lester Hill's cipher (1929) uses linear algebra. The plaintext is split into blocks of m letters, each treated as a vector P. The key is an m × m matrix K. Encryption: $C = K \cdot P \bmod 26$. Decryption: $P = K^{-1} \cdot C \bmod 26$. The key matrix must be invertible mod 26, which requires $\gcd(\det K, 26) = 1$.

Worked example (m = 2). K = [[3, 3], [2, 5]]. det K = 3 × 5 - 3 × 2 = 9, and gcd(9, 26) = 1, so K is valid. Plaintext HELP becomes the vectors (7, 4) and (11, 15).

- (7, 4): 3 × 7 + 3 × 4 = 33 ≡ 7; 2 × 7 + 5 × 4 = 34 ≡ 8. Result (7, 8) = HI.
- (11, 15): 3 × 11 + 3 × 15 = 78 ≡ 0; 2 × 11 + 5 × 15 = 97 ≡ 19. Result (0, 19) = AT.

Ciphertext: HIAT.

Decryption. The inverse of the determinant, $9^{-1} \bmod 26$, is 3 (since 9 × 3 = 27 ≡ 1). The adjugate of K is [[5, -3], [-2, 3]]. So $K^{-1} = 3 \times [[5, -3], [-2, 3]] \bmod 26 = [[15, 17], [20, 9]]$. Check (7, 8): 15 × 7 + 17 × 8 = 241 ≡ 7 and 20 × 7 + 9 × 8 = 212 ≡ 4, giving (7, 4) = HE.

Strength and weakness: it hides single-letter frequencies completely (with m = 2 it hides digram patterns partly), so it resists ciphertext-only attack well. But it is linear, so a known-plaintext attack (m plaintext-ciphertext pairs of blocks) lets the attacker solve for K.

1.5 Polyalphabetic ciphers: Vigenère

Several monoalphabetic substitutions are used in turn, selected by a keyword. For the Vigenère cipher: $C_i = (P_i + K_{i \bmod m}) \bmod 26$, where m is the keyword length and the keyword repeats.
Worked example. Plaintext ATTACKATDAWN, keyword LEMON.

Plaintext	A	T	T	A	C	K	A	T	D	A	W	N
Key	L	E	M	O	N	L	E	M	O	N	L	E
Ciphertext	L	X	F	O	P	V	E	F	R	N	H	R

Ciphertext: LXFOPVEFRNHR. Note that the three Ts in the plaintext become X, F, F and the A's become L, O, E, N, so one letter maps to several different letters.
Cryptanalysis: the Kasiski method finds repeated sequences in the ciphertext. The distances between repeats are likely multiples of the key length. Once the key length m is known, the ciphertext is split into m columns and each is broken by frequency analysis as a Caesar cipher. The index of coincidence gives another estimate of m.
Autokey variants use the plaintext itself as part of the key to avoid repetition of the key.

1.6 One-time pad (Vernam cipher)

The key is a truly random stream as long as the message, used only once. Encryption: $C_i = P_i + K_i \bmod 26$ (or XOR for bits).
Example: HELLO with key XMCKL: H+X = E, E+M = Q, L+C = N, L+K = V, O+L = Z, giving EQNVZ.
It is **unconditionally secure**: any ciphertext can correspond to any plaintext of the same length, depending on the key, so there is no information to exploit. Practical drawbacks: generating long truly random keys, distributing them securely, and the rule that the key must never be reused (reuse leaks the XOR of two plaintexts).

2. Transposition Ciphers

Letters are kept but their positions are permuted. Because the letters are unchanged, the ciphertext has exactly the same letter frequencies as ordinary language. That is the clue a cryptanalyst uses to recognize a transposition cipher.

2.1 Rail fence cipher

The plaintext is written diagonally in a zigzag across a chosen number of rows (rails) and read off row by row.
Example, 2 rails, plaintext MEETMEAFTER: letters in positions 1, 3, 5, ... go to rail 1 (M, E, M, A, T, R) and positions 2, 4, 6, ... go to rail 2 (E, T, E, F, E). Ciphertext: MEMATRETEFE. The key is the number of rails, so the cipher is easy to break.

2.2 Row-column (columnar) transposition

The plaintext is written in rows of fixed length under a keyword (the key defines the column order). Columns are then read in the order given by the alphabetical order of the keyword letters (or by the numeric key).
Example: numeric key 3 1 4 2 and plaintext ATTACKATDAWN (12 letters, 3 rows of 4):

Key	3	1	4	2
	A	T	T	A
	C	K	A	T
	D	A	W	N

Read column 1 (the one under key 1): TKA; then key 2: ATN; key 3: ACD; key 4: TAW.
Ciphertext: TKAATNACDTAW.
To decrypt, the receiver computes the column length (message length divided by key length), writes the ciphertext into the columns in key order, and reads the rows.
A **double transposition** applies the process twice with different keys and is considerably harder to break.

2.3 Product ciphers

Combining substitution and transposition in succession produces a cipher stronger than either alone. A product cipher is the link between classical and modern ciphers: DES and AES repeat rounds of substitution and permutation.

3. Cryptanalysis

3.1 Types of attack by information available

Attack	Information the attacker has
Ciphertext only	Algorithm and ciphertext. Hardest for the attacker.
Known plaintext	Plus some plaintext-ciphertext pairs (for example standard headers).
Chosen plaintext	Attacker can choose plaintext and get its ciphertext.
Chosen ciphertext	Attacker can choose ciphertext and get its plaintext.
Chosen text	Both chosen plaintext and chosen ciphertext.

3.2 Methods

- **Brute force**: try every key. Defeated by a large key space (at least 128 bits for symmetric ciphers today).
- **Statistical (frequency) analysis**: uses letter, digram and word statistics. Breaks classical ciphers.
- **Mathematical attacks on modern ciphers**: differential cryptanalysis (studies how differences in plaintext pairs propagate to ciphertext) and linear cryptanalysis (finds linear approximations of the cipher).
- **Implementation attacks**: timing, power analysis and cache attacks target the implementation rather than the algorithm.

A cipher is **unconditionally secure** if the ciphertext does not contain enough information to determine the plaintext, regardless of resources. It is **computationally secure** if breaking it costs more than the value of the data or takes longer than the data stays useful.

4. Steganography

Cryptography hides the *meaning* of a message. Steganography hides the *existence* of the message. If the message is discovered it is readable, so steganography is often used together with encryption.

4.1 Classical techniques

- **Character marking**: selected letters are overwritten in pencil or lightly marked so they spell out a message.
- **Invisible ink**: substances that show only after heating or chemical treatment.
- **Pin punctures**: small holes over selected letters, visible when held to light.
- **Typewriter correction ribbon**: text typed with a used ribbon that is visible only under strong light.
- **Acrostics**: the first (or nth) letters of each word or line spell the hidden message.

4.2 Digital techniques

- **Least significant bit (LSB) insertion**: the lowest bit of each pixel color value (or audio sample) is replaced with a message bit. For example, a pixel value 10110100 carrying the bit 1 becomes 10110101. The change is too small for the eye to notice. A 24-bit image of N pixels can hide about 3N bits.
- Embedding in unused or reserved fields of file headers, network protocol headers, or slack space.
- Using spacing, font changes or invisible characters in text.
- Embedding in transform-domain coefficients (for example JPEG DCT) so the message survives compression.

4.3 Evaluation

- **Advantages**: hides the fact that communication is taking place, and does not attract attention the way ciphertext does.
- **Drawbacks**: needs a large cover for a small payload (high overhead). Once the method is known, the message can be extracted. Steganalysis can detect statistical anomalies.

Related idea: digital watermarking embeds ownership information to show copyright, and it must be robust to removal.

Part B: Modern Block Ciphers

5. Stream Ciphers and Block Ciphers

Point	Stream cipher	Block cipher
Data unit	One bit or one byte at a time	A fixed-size block (64 or 128 bits) at a time
Operation	Plaintext XORed with a pseudorandom keystream generated from the key	Each block is mapped by the key into a ciphertext block of the same size
Memory	Keystream state changes continuously	Memoryless within a block (modes add chaining)
Speed and use	Very fast, good for real-time streams and hardware with little memory	General purpose files and messages, network protocols
Examples	RC4, Salsa20 and ChaCha20	DES, 3DES, AES
Rule for safe use	Never reuse a key and nonce combination	Use a proper mode of operation

5.1 Stream cipher design

The ideal is a one-time pad. A practical stream cipher replaces the truly random pad with a pseudorandom keystream from a key. The keystream must have a long period, appear random, and depend on a long enough key (at least 128 bits).
RC4 outline: it keeps a 256-byte state array S. The key-scheduling step initializes S as 0..255 and permutes it using the key. The output step repeatedly swaps entries of S and outputs one byte, which is XORed with the plaintext byte. RC4 has known biases and has been dropped from TLS, so it is a classroom example today.

6. Block Cipher Principles

6.1 Ideal block cipher

An n-bit block cipher maps each of the 2^n possible plaintext blocks to a distinct ciphertext block (a permutation). The ideal cipher would be a table with a different arbitrary permutation for each key, but the key would then need $n \times 2^n$ bits, which is impractical (for n = 64 that is about 10^{21} bits). Practical ciphers approximate the ideal using a smaller key and a structured algorithm.

6.2 Shannon's confusion and diffusion

Claude Shannon (1949) proposed two properties for strong ciphers, to frustrate statistical attack.

- **Diffusion**: the statistical structure of the plaintext is dissipated into long-range statistics of the ciphertext. Each plaintext bit should affect many ciphertext bits, and each ciphertext bit should depend on many plaintext bits. It is achieved by **permutation** and mixing operations.
- **Confusion**: the relationship between the key and the ciphertext is made as complex as possible, so that the key cannot be deduced from the ciphertext even if the attacker holds statistics of the ciphertext. It is achieved by complex **substitution** (S-boxes).
- **Avalanche effect**: a small change in plaintext or key (even one bit) should change about half of the ciphertext bits.

6.3 Feistel cipher structure

Horst Feistel proposed alternating substitutions and permutations (a product cipher) in a structure that is still used in DES and many other ciphers.

Encryption

1. Split the 2w-bit plaintext block into a left half L_0 and a right half R_0 of w bits each.
2. For each round i = 1 to n: $L_i = R_{i-1}$ and $R_i = L_{i-1}$ XOR $F(R_{i-1}, K_i)$.
3. After the final round swap the halves and join them: the ciphertext is (R_n, L_n).

F is the **round function** and K_i is the subkey for round i, derived from the main key by a key schedule.

Decryption uses the same structure and the same F, with the subkeys used in reverse order K_n, ..., K_1. It works because each round undoes itself: given L_i and R_i, the previous values are $R_{i-1} = L_i$ and $L_{i-1} = R_i$ XOR $F(L_i, K_i)$. **F need not be invertible**, which is the big attraction of the structure.

Design parameters

- Block size: larger gives more security but slower speed (64 bits was typical, 128 bits is standard now).
- Key size: larger gives more resistance to brute force (128 bits or more today).
- Number of rounds: more rounds improve security (typical values are 16 for DES).
- Subkey generation algorithm: should be complex so that subkeys are hard to deduce.
- Round function F: greater complexity means greater resistance to cryptanalysis.
- Fast software encryption and decryption, and ease of analysis for confidence in the design.

7. Data Encryption Standard (DES)

7.1 History

DES was developed by IBM from the Lucifer cipher and adopted by the US National Bureau of Standards as FIPS 46 in 1977. It was the dominant symmetric cipher for about two decades. It was replaced by AES because its 56-bit key became too short.

7.2 Parameters

- Block size 64 bits.
- Key 64 bits as given, of which 8 are parity bits, so the effective key is 56 bits.
- 16 rounds of a Feistel structure, each using a 48-bit round subkey.

7.3 Overall structure

1. The 64-bit plaintext passes through the **initial permutation (IP)**, which rearranges the bits according to a fixed table.
2. The result is split into 32-bit halves L_0 and R_0.
3. Sixteen Feistel rounds are applied: $L_i = R_{i-1}$ and $R_i = L_{i-1}$ XOR $F(R_{i-1}, K_i)$.
4. The halves are swapped (pre-output is R_{16} L_{16}).
5. The **final permutation (IP^{-1})**, the inverse of IP, gives the 64-bit ciphertext.

IP and IP^{-1} add no cryptographic strength (they are fixed and public). They were included for hardware loading reasons.

7.4 Round function F(R, K)

1. **Expansion permutation (E)**: the 32-bit R is expanded to 48 bits by duplicating 16 of its bits. This lets one bit affect two S-boxes, which increases diffusion.
2. **Key mixing**: the 48 bits are XORed with the 48-bit round subkey K_i.
3. **Substitution by 8 S-boxes**: the 48 bits are split into eight groups of 6 bits. Each group is the input of one S-box (S1 to S8), which outputs 4 bits, giving 32 bits in all. For each S-box, the outer two bits (first and last) select the row (0 to 3) and the middle four bits select the column (0 to 15). The S-boxes are the only nonlinear part of DES and provide confusion.
4. **Permutation (P)**: the 32 output bits are permuted, so the output of each S-box influences the inputs of several S-boxes in the next round (diffusion).

S-box example: for S1 with the 6-bit input 011011, the row is 01 (row 1) and the column is 1101 (column 13). The table entry in row 1, column 13 of S1 is 5, so the output is 0101.
S-box design properties: each row is a permutation of 0 to 15; no S-box is a linear or affine function of its inputs; changing one input bit changes at least two output bits. These were chosen to resist differential cryptanalysis.

7.5 Key schedule

1. The 64-bit key goes through **Permuted Choice 1 (PC-1)**, which discards the 8 parity bits and permutes the other 56 bits.
2. The 56 bits are split into two 28-bit halves C_0 and D_0.
3. In each round, C and D are each rotated left by 1 or 2 bits. The numbers of left shifts for rounds 1 to 16 are: 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1 (28 positions in total, so $C_{16} = C_0$).
4. **Permuted Choice 2 (PC-2)** selects 48 of the 56 bits (24 from C and 24 from D) to form K_i.

7.6 Decryption

The same algorithm is used with the subkeys in reverse order (K_{16} first, K_1 last). This is a direct consequence of the Feistel structure.

7.7 Avalanche effect

A change of one bit in the plaintext or in the key changes about half of the ciphertext bits after only a few rounds. That is a desirable property of DES.

8. Strength of DES

- **Key size**: with 2^{56} (about 7.2×10^{16}) possible keys, brute force became practical. In 1998 the Electronic Frontier Foundation built a special-purpose machine (the DES Cracker, built for less than 250,000 US dollars) that recovered a key in a few days, and in 1999 a combined effort cut this to under a day. Modern FPGAs and clusters do it faster and cheaper. DES is therefore considered insecure.
- **Differential cryptanalysis** (Biham and Shamir, 1990): analyzes how chosen plaintext pairs with a particular difference propagate. It needs about 2^{47} chosen plaintexts to break full DES. The S-box design (known to IBM and the NSA from the 1970s) was in fact tuned to resist this.
- **Linear cryptanalysis** (Matsui, 1993): finds linear approximations of the cipher. Needs about 2^{43} known plaintext-ciphertext pairs. Both attacks are theoretical because of the amount of data needed.
- **Weak keys**: there are 4 weak keys for which all subkeys are identical, so encryption and decryption are the same operation ($E_K(E_K(P)) = P$). There are also 6 pairs of **semi-weak keys** (12 keys) where one key of a pair undoes the other. These keys are easily avoided.
- **Complementation property**: if C = E(K, P), then the bitwise complement of C equals E(complement of K, complement of P). This reduces the brute-force effort by a factor of two for chosen plaintext.
- **Timing attacks**: depend on the implementation rather than the algorithm.
- Concern in the 1970s about hidden trapdoors in the S-boxes has not been substantiated.

Conclusion: the Feistel structure and S-boxes of DES are sound. The key length is the weakness, which is why Triple DES, then AES, replaced it.

9. Multiple Encryption and Triple DES

9.1 Double DES and the meet-in-the-middle attack

Double encryption $C = E(K_2, E(K_1, P))$ appears to give a 112-bit key. It does not. The attacker, who knows one plaintext-ciphertext pair, computes $X = E(K_1, P)$ for all 2^{56} values of K_1 and stores the results in a table. Then for each K_2 they compute $D(K_2, C)$ and look for a match in the table. A match gives a likely key pair. The effort is about 2^{57} operations ($2^{56} + 2^{56}$) and 2^{56} storage, not 2^{112}. A second plaintext-ciphertext pair eliminates false matches.

9.2 Triple DES (3DES)

Encryption: $C = E(K_3, D(K_2, E(K_1, P)))$. Decryption: $P = D(K_1, E(K_2, D(K_3, C)))$. This is the encrypt-decrypt-encrypt (EDE) form.

- **Three-key 3DES**: three independent 56-bit keys, giving a 168-bit key. The effective strength against meet-in-the-middle is about 112 bits.
- **Two-key 3DES**: $K_3 = K_1$, giving a 112-bit key.
- The middle step is a decryption so that if $K_1 = K_2 = K_3$ the result equals single DES. This keeps compatibility with DES systems.
- Drawbacks: about three times slower than DES. The 64-bit block size makes it vulnerable to birthday-type attacks when large amounts of data are encrypted under one key (Sweet32 attack). NIST has deprecated and phased out 3DES, so AES is the standard choice.

10. Advanced Encryption Standard (AES)

10.1 Background

In 1997 NIST called for a replacement for DES, requiring a symmetric block cipher with a 128-bit block and 128, 192 and 256-bit keys that would be efficient in both hardware and software. Fifteen candidates were evaluated, and the Belgian design **Rijndael** (Joan Daemen and Vincent Rijmen) was selected in 2000. It was published as FIPS 197 in November 2001.

10.2 Parameters

Version	Key size	Nk (key words)	Block size	Nb (state columns)	Rounds Nr	Round key words
AES-128	128 bits	4	128 bits	4	10	44
AES-192	192 bits	6	128 bits	4	12	52
AES-256	256 bits	8	128 bits	4	14	60

Nr = Nk + 6, and the number of round-key words is 4 × (Nr + 1).

10.3 Structure

- AES is **not a Feistel cipher**. It is a substitution-permutation network that processes the **whole block** in each round.
- The 16 bytes of the block are arranged in a 4 × 4 byte matrix called the **state**, filled column by column.
- Bytes are treated as elements of the finite field $GF(2^8)$ with the irreducible polynomial $x^8 + x^4 + x^3 + x + 1$. Addition is XOR.

Encryption outline

1. Initial transformation: AddRoundKey with round key 0.
2. For rounds 1 to Nr - 1: SubBytes, ShiftRows, MixColumns, AddRoundKey.
3. Final round (round Nr): SubBytes, ShiftRows, AddRoundKey. **MixColumns is omitted**.

10.4 The four round operations

SubBytes (confusion): each byte of the state is replaced independently using a fixed 16 × 16

table (the S-box). The high nibble of the byte selects the row and the low nibble the column. The S-box is built from the multiplicative inverse in $GF(2^8)$ (with 00 mapped to 00) followed by an affine transformation. It is a nonlinear, invertible, and has no fixed points. Examples: S(00) = 63, S(53) = ED.

ShiftRows (diffusion between columns): row 0 is not shifted; row 1 is rotated left by 1 byte; row 2 by 2 bytes; row 3 by 3 bytes. After this, the four bytes of each column come from four different columns, so the next MixColumns mixes bytes from the whole state.

MixColumns (diffusion within columns): each column is multiplied, as a vector over $GF(2^8)$, by the fixed matrix

02	03	01	01
01	02	03	01
01	01	02	03
03	01	01	02

Multiplication by 02 is a left shift by one bit, followed by an XOR with 1B if the top bit was 1 (this is called xtime). Multiplication by 03 is multiplication by 02 XORed with the byte itself. Test example: the column (DB, 13, 53, 45) becomes (8E, 4D, A1, BC).

AddRoundKey: the 128-bit state is XORed with the 128-bit round key. It is the only step that uses the key, and it is its own inverse.

10.5 Key expansion

The cipher key is expanded into a key schedule of 4 × (Nr + 1) words w[0], w[1], ... The first Nk words are the cipher key itself. For each later word w[i]:

- If i is not a multiple of Nk: w[i] = w[i - 1] XOR w[i - Nk].
- If i is a multiple of Nk: w[i] = w[i - Nk] XOR SubWord(RotWord(w[i - 1])) XOR Rcon[i / Nk].
- For AES-256 only, when i mod Nk = 4, SubWord is applied to w[i - 1] before the XOR (without rotation or Rcon).

RotWord rotates the 4 bytes of a word one place left. SubWord applies the S-box to each of the 4 bytes. Rcon is a round constant word whose first byte is 01, 02, 04, 08, 10, 20, 40, 80, 1B, 36 for successive rounds (and zeros in the other bytes). Each round takes four consecutive words as its round key.

10.6 Decryption

Decryption applies the inverse operations in reverse order, with round keys in reverse: InvShiftRows (rotate right), InvSubBytes (inverse S-box), AddRoundKey, InvMixColumns (multiply by the inverse matrix with entries 0E, 0B, 0D, 09). Since AES is not a Feistel cipher, the inverse operations are needed separately. The decryption order for a round is InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.

10.7 Strength and status

- The full AES has no practical attack. The best published key-recovery attacks reduce the work for AES-128 only by a small factor (about two bits), which is still far beyond feasible.
- Practical weaknesses are in implementations (cache-timing and other side channels),

which is why hardware instructions (AES-NI) and constant-time code are used.
- AES is the current standard for symmetric encryption and is used in TLS, Wi-Fi (WPA2 and WPA3), disk encryption and VPNs.

11. DES versus AES

Point	DES	3DES	AES
Structure	Feistel	Three DES passes (EDE)	Substitution-permutation network
Block size	64 bits	64 bits	128 bits
Key size	56 bits	112 or 168 bits	128, 192 or 256 bits
Rounds	16	48 (3 × 16)	10, 12 or 14
Processing per round	Half the block	Half the block	Whole block
Nonlinear step	8 S-boxes (6 to 4 bits)	Same	One S-box applied to every byte
Speed	Moderate	Slow	Fast, with hardware support
Status	Insecure, obsolete	Phased out	Current standard

12. Modes of Operation

A block cipher encrypts one block. A **mode of operation** specifies how to encrypt messages longer than one block, and how to deal with a final partial block (padding). NIST SP 800-38A defines five main modes.

Mode	Encryption	Properties
ECB (Electronic Codebook)	$C_i = E(K, P_i)$	Blocks encrypted independently. Identical plaintext blocks give identical ciphertext blocks, so patterns show. Parallel, no error propagation. Suitable only for very short data such as a key.
CBC (Cipher Block Chaining)	$C_i = E(K, P_i \text{ XOR } C_{i-1})$, with $C_0 = IV$	Chaining hides patterns. Needs a random, unpredictable IV. Encryption is sequential, decryption can be parallel. A bit error damages two blocks.

Mode	Encryption	Properties
CFB (Cipher Feedback)	$C_i = P_i$ XOR $E(K, C_{i-1})$ (the first input is the IV)	Acts as a self-synchronizing stream cipher. Can work on units smaller than a block. Only the encrypt direction of the block cipher is used.
OFB (Output Feedback)	$O_i = E(K, O_{i-1})$ with $O_0 = IV$; $C_i = P_i$ XOR O_i	Keystream independent of the data and can be precomputed. A bit error affects only that bit. The IV must never repeat for one key.
CTR (Counter)	$C_i = P_i$ XOR $E(K,$ nonce $\|\|$ $\text{counter}_i)$	Fully parallel, random access to any block, hardware efficient. The nonce and counter must never repeat under one key.

- **Padding**: ECB and CBC need the last block padded to full size (for example PKCS#7 padding).
- None of these modes gives integrity. Authenticated modes such as GCM combine CTR encryption with an authentication tag, and are recommended in new designs.

13. Likely Exam Questions and Answer Outlines

1. **Explain the Feistel cipher structure with a diagram.** Draw the L and R halves with the round function F and the subkey K_i. Write the two round equations, explain that decryption reverses the subkey order and that F need not be invertible, and list the design parameters.
2. **Explain DES with the round function and the key schedule.** Parameters, IP, 16 rounds, expansion, key XOR, S-boxes with the row and column rule, P permutation, final permutation, PC-1, shifts and PC-2.
3. **Discuss the strength of DES.** Key size and the 1998 and 1999 attacks, differential and linear cryptanalysis, weak keys, complementation property.
4. **Why is double DES not used? Explain Triple DES.** The meet-in-the-middle attack, then the EDE form, key sizes and backward compatibility.
5. **Explain AES with all four transformations.** Parameters table, state matrix, the four operations, no MixColumns in the last round, key expansion and decryption.
6. **Differentiate between DES and AES.** Use the table in Section 11.
7. **Define confusion and diffusion.** Shannon's definitions, how substitution and permutation achieve them, and the avalanche effect.
8. **Differentiate stream and block ciphers. Explain the modes of operation.** Use the tables in Sections 5 and 12 and draw the CBC diagram.
9. **Numericals**: encrypt and decrypt with Caesar, Playfair, Hill (2 × 2), Vigenère, rail fence and columnar transposition. Practice each at least three times.

10. **Write short notes on steganography.** Definition, classical and digital techniques with the LSB example, advantages and drawbacks.

Diagrams to practice

- Feistel network (two rounds, with the final swap).
- DES overall flow, one DES round, and the key schedule.
- AES encryption flow, state matrix, ShiftRows and MixColumns.
- CBC and CTR mode encryption.

Sources consulted

- NIST FIPS 197, Advanced Encryption Standard for AES parameters and structure.
- Raj Jain, Washington University in St. Louis, AES lecture slides for the number of rounds, key expansion and the round steps.
- University of Oxford DES lecture notes for the S-box row and column rule, key schedule and the meet-in-the-middle attack on double DES.
- William Stallings, Cryptography and Network Security, and Atul Kahate, Cryptography and Network Security (course text), for the classical ciphers, Feistel structure, DES and modes of operation.